# Profile IAM published events: `UserProfileChanged` and `UserProfileUpdated`

CONTENTS


1. Summary 2
2. UserProfileChanged 2
   2.1 How it is published 2
   2.2 Feature flag 3
   2.3 Every place a change is recorded (and so published) 3
   2.4 Notes on the payload 6
3. UserProfileUpdated 6
   3.1 How it is published 6
   3.2 Notes on the payload 6
4. How consumers use them today 7
   4.1 Wallet (UserProfileChanged) 7
   4.2 Remittances (UserProfileChanged and UserProfileUpdated) 8
   4.3 Observations 9
5. Findings 9


Source reviewed: `gkone-user-profile-iam` @ `main` (58092ef), contracts from `GKOne.Messaging` 6.6.3 and `GKOne.UserProfile.SharedLibrary` 4.3.0 (decompiled from the pinned packages), MassTransit 8.5.7. Paths below are relative to `src/gkone-user-profile-iam.web/`.

| File | Contents |
|---|---|
| UserProfileChanged.schema.json | JSON Schema (2020-12) of the message body |
| UserProfileChanged.sample.json | Sample message body (validated against the schema) |
| UserProfileUpdated.schema.json | JSON Schema (2020-12) of the message body |
| UserProfileUpdated.sample.json | Sample message body (validated against the schema) |

Samples are the `message` object only. On the wire MassTransit wraps it in its envelope (`application/vnd.masstransit+json`: `messageId`, `messageType`, `sourceAddress`, `headers`, `message`). Values are illustrative, not captured from a running environment.

# 1. Summary

Both events go to the same Azure Service Bus topic, `AzureServiceBus:ProfileIAmTopicName` (`Registrations/MassTransitConfiguration.cs:166–170`). They have exactly one publish call each:

| Event | Publish call | Called from | Delivery |
|---|---|---|---|
| `UserProfileChanged` | `Services/ServiceBus/UserProfileChangedPublisher.cs:62` | Only `ProfileChangeOutboxDispatcher.cs:250`. `ProfileChangeOutboxArchitectureTests` fails the build if anything else depends on `IUserProfileChangedPublisher`. | Transactional outbox, at least once, `MessageId = eventId` |
| `UserProfileUpdated` | `Services/Kyc/KycPublisherService.cs:22` | Only `KycService.CompleteKycAsync` (V1, `KycService.cs:196`) and `CompleteKycAsync` (V2, `KycService.cs:267`) | Direct publish after the write, errors logged and swallowed, no retry |

`UserProfileUpdated` is therefore a KYC-completion event only. It is not published for any other profile update, despite its name.

Consumers: Wallet reads `UserProfileChanged` only, and Remittances reads both (section 4). Both match on the message Subject, which Profile IAM sets to the type name (`MessageFilters.cs:81`).

A KYC completion publishes both events: `RecordProfileChange(KycCompletion)` feeds the outbox, and `CompleteKycAsync` then publishes `UserProfileUpdated` itself. `KycProfileUpdateEventParityTests` pins this, noting the legacy event "remains available to other production consumers".

# 2. `UserProfileChanged`

## 2.1 How it is published

1. A service mutates the `GkOneUser` and calls `user.RecordProfileChange(changeSource, changedFields)` before the Cosmos write. This appends a `PendingProfileChange` (id, source, fields, recorded-at, correlation id) to the same document, so the record commits with the write. An empty field set records nothing, and a repeat of the same source and field set is not added twice.
2. `ProfileChangeOutboxHostedService` sweeps every `SweepIntervalSeconds` (10 s), holds a shared lease, and calls `ProfileChangeOutboxDispatcher.DispatchAsync`.

3. The dispatcher reads each profile with its ETag, calls `UserProfileChangedPublisher.TryPublishAsync` per pending change in recorded order (the first refusal stops the rest), then clears the published ones with an ETag-guarded write. A failed clear republishes under the same `eventId` on the next sweep.
4. The event is built by `ProfileChangeEventMapper.MapToEvent`. The snapshot is the document as it is at publish time, and `occurredAtUtc` is that document's `UpdatedAtUtc`.

## 2.2 Feature flag

The hosted service does nothing unless `EnableProfileChangeOutboxDispatch` is on:

| Environment | Value |
|---|---|
| base `appsettings.json`, **Production** | `false` |
| Development, QA, UAT | `true` |

With the flag off, changes are still recorded on the profile documents and accumulate as pending, but no `UserProfileChanged` reaches the topic. At the time of this review, Production publishes none.

## 2.3 Every place a change is recorded (and so published)

Each row is a `RecordProfileChange` call. `changedFields` use the `ProfileChangedFieldNames` vocabulary from `GKOne.Messaging`.

| changeSource | changedFields | Recorded in | Triggered by |
|---|---|---|---|
| `EmailUpdate` | `email` | `UsersService.cs:344` | OTP-verified email change (`OtpService.cs:612`) |
| `PhoneNumberUpdate` | `phoneNumber` | `UsersService.cs:523` (`ApplyVerifiedPhoneChangeAsync`) | OTP-verified phone change (`OtpService.cs:653`); needs `EnablePhoneNumberProfileMigration` (off in Production) |
| `PhoneNumberUpdate` | `phoneNumber` | `UsersService.cs:496` (`UpdatePhoneNumber`) | **No production caller** (interface and definition only) |
| `PrimaryAddressUpdate` | `address` | `UsersService.cs:100` | `POST /user/primaryAddress` |

| changeSource | changedFields | Recorded in | Triggered by |
|---|---|---|---|
| `OccupationalLevelUpdate` | `occupationalLevel` | `UsersService.cs:383` | `PUT api/Users/UpdateOccupationalLevel`, only when the value changed |
| `AdminUserDetailsPatch` | the fields the PATCH changed (`trn`, `motherName`, ...) | `UsersService.cs:426` | Admin `PATCH` on `UsersController.UpdateUserDetails` |
| `KycStatusUpdate` | `kycStatus` | `UsersService.cs:552` | `PUT api/Kyc/UpdateKycStatus`, only when the status moved |
| `TaxRegistrationNumberUpdate` | `trn` | `UsersService.cs:735` | `PUT api/Kyc/updateTrn`, only when the TRN was blank |
| `IdentificationDocumentUpdate` | `identificationDocument` | `UsersService.cs:801` | `PUT api/Users/SetMostRecentIdentificationDocumentIssueDate` |
| `IdentificationDocumentUpdate` | `identificationDocument` | `CustomerIdentificationService.cs:174` | Sign-in, when a CMS document is appended |
| `IdentificationDocumentUpdate` | `identificationDocument` | `AccountService.cs:691` | `SignInAsync` / `SignInB2CAsync` document backfill |
| `CmsAddressSync` | `address` | `UsersService.cs:864` | Sign-in (`AccountService.cs:778`) address sync from CMS |
| `EntraProfileSync` | detected diff vs. pre-sync snapshot | `AccountService.cs:422` | Entra sign-in profile sync, only on a real change |
| `CustomerNameVariationsSync` | `customerNameVariations` | `AccountService.cs:741` | Sign-in, only when the variations changed |
| `KycStatusReset` | `kycStatus` | `KycService.cs:137` | `PUT api/Kyc/resetKycStatus`, `PATCH api/Kyc/admin-reset-kyc-status`, `PATCH api/Kyc/admin-reset-kyc-status-by-email` |
| `KycCompletion` | `address`, `trn`, `externalWalletReferenceNo` | `KycService.cs:190` (V1), `KycService.cs:261` (V2) | `POST api/Kyc/completeInfo`, `POST /api/v2/Kyc/completeInfo` |

| changeSource | changedFields | Recorded in | Triggered by |
|---|---|---|---|
| `CreditCardApplicationProfileUpdate` | `title`, `occupation`, `maritalStatus`, `monthlyIncome`, `monthlyExpense`, `employmentStatus`, `mothersMaidenName`, `address` | `KycService.cs:335` | `POST api/Kyc/UpdateFromCreditCardApplication` |
| `CmsProfileValidation` | detected diff vs. pre-validation snapshot | `KycService.cs:380` | `PUT api/Kyc/v1/validateCustomerProfile` |
| `PrepaidCardRegistration` | `prepaidCardInfo` | `CardService.cs:56`, `CardService.cs:100` (mock flow) | `POST api/Card/choose-pick-up-location/{locationId}` |
| `PrepaidCardRegistration` | `prepaidCardInfo`, `kycStatus` | `OpenLoopCardProfileUpdateService.cs:66` | Open-loop card profile-update subscription; needs `EnableOpenLoopCardProfileUpdateQueue` (off in Production) |
| `PrepaidCardReplacement` | `prepaidCardInfo` | `CardService.cs:167` (mock flow), `CardService.cs:200` | `POST api/Card/replace-card` |
| `OndatoIdentityVerification` | detected diff vs. pre-webhook snapshot | `OnDatoWebhookHandlingService.cs:483`, `Ondato/WebhookHandling/Context/OndatoWebhookHandlingContext.cs:224` | Ondato webhook (`KycController.Webhook`) |
| `OndatoIdentityVerification` | `kycStatus` | `Ondato/WebhookHandling/Strategy/OndatoApprovedWebhookStrategy.cs:285` | Ondato approval that flips another profile with the same TRN to `TRN_DUPLICATED` |
| `IdentityConflictResolution` | detected diff vs. pre-resolution snapshot | `IdentityConflictService.cs:704` (called at 728, 837, 923) | Identity-conflict resolution and merge; needs `EnableIdentityConflictResolution` (off in Production and UAT) |

"Detected diff" sources use `ProfileChangedFieldDetector.DetectChangedFields`, so their `changedFields` depend on what actually changed on that call.

Deliberate non-publishers are listed in `gkone-iam.Tests/Architecture/ProfileWritePathPublishCoverageCatalog.cs` (terms acceptance, passcode token, account-deletion request, KYC session bootstrap, sign-up).

### 2.4 Notes on the payload

- `changeSource` and `changedFields` are plain strings on the contract, so the lists in the schema are a snapshot; consumers should tolerate new values.
- `correlationId` is the 32-character W3C trace id of the request that recorded the change. It is not a full `traceparent`.
- `profile.paymentReferences.prepaidCardInfo.maskedCardNumber` keeps the first 6 and last 4 digits. The full card number is never in this event.
- `status`, `kycStatus`, `kycTier`, `cmsStatus` and `identificationDocument.documentType` come from `EnumExtensions.DisplayName`. The samples assume that is the enum name. I could not confirm that from the code, so check it against a captured message.
- `changedFields` can name `motherName`, `title`, `maritalStatus`, `monthlyIncome`, `monthlyExpense`, `employmentStatus` or `mothersMaidenName`, none of which has a property in `profile`.

## 3. `UserProfileUpdated`

### 3.1 How it is published

`KycService.CompleteKycAsync` (V1 and V2) updates the user, then builds the event with `CreateProfileUpdatedEvent` (`KycService.cs:276`): a Newtonsoft deep copy of the persisted `GkOneUser` with `PendingProfileChanges` cleared. It is handed to `KycPublisherService.PublishProfileUpdatedOrLogError`, which publishes it with `MessageId = event.Id` and, on any exception, logs `"Failed to publish for user {UserId}"` and returns normally.

Triggers: `POST api/Kyc/completeInfo` and `POST /api/v2/Kyc/completeInfo`. No feature flag gates it.

### 3.2 Notes on the payload

- `id` is a new Guid on every publish, so unlike `eventId` on `UserProfileChanged` it cannot be used to dedupe a redelivery. Dedupe on the user's `updatedAtUtc` if needed.
- It is the whole `GkOneUser` document. Everything the model serializes goes onto the topic if populated, including `password`, `hash`, `passcodeVerificationToken`, `emailVerification.token`, `creditCardInfo.cardNumber` (full, unmasked), `wallet.cardNumber` and `wallet.cvv`. The samples show these null or illustrative. Which of them are populated in real documents was not checked.
- Enums are integers (see the `KycStatus` etc. descriptions in the schema), decimals are strings, property names are camelCase. This follows from MassTransit's `System.Text.Json` defaults, which `MassTransitConfiguration` does not override apart from hiding the outbox fields. I confirmed it from the configuration and the MassTransit source, not from a captured message.

- The schema is derived from the package model, so it describes what can be serialized. Types it does not expand (employer address, T&C type enum, supporting-document internals) are left loose.

# 4. How consumers use them today

Reviewed: `gkone-wallet` (branch `feature/38064-wallet-platform-modernisation-net-10-native-cosmos-db-access-cleaner-boundaries-2` @ a28f5ca) and `gkone-remittances` (`main` @ e03fee0). `gkone-order-management` also references both events and was not reviewed.

Both consumers read the same topic, `sbt-profile-iam-{env}`, through their own subscription, using the Service Bus client directly (not MassTransit). Both select by message **Subject**, which Profile IAM sets in `Services/ServiceBus/MessageFilters.cs` as `SetLabel(typeof(T).Name)`, so `UserProfileChanged` and `UserProfileUpdated` are also the subject names. Both unwrap the MassTransit `message` envelope, deserialize with Newtonsoft (unknown members ignored, case-insensitive), and treat `eventId` / `occurredAtUtc` as ordering keys.

| | Wallet | Remittances |
|---|---|---|
| Consumes | `UserProfileChanged` only. Any other subject, including `UserProfileUpdated`, is logged and completed without applying. | Both |
| Subscription | `sbts-wallet-user-profile-{env}` | `sbts-remittances-profile-updates-{env}` |
| Entry point | `ProfileIamTopicSubscriberService` | `RemittanceUserProfileEventSyncService` |
| Subscription rules | Not provisioned by the repo. The ops doc says to narrow the rule to the `UserProfileChanged` subject before promotion. | Declares `sys.Label = 'UserProfileChanged'` and `sys.Label = 'UserProfileUpdated'` |
| Feature flag | `WalletUserProfileEventSyncEnabled`, read once at startup. `true`: Development, QA, UAT, ProductionStaging. `false`: base, Production. | `EnableRemittanceUserProfileEventSync`, re-read every minute. `true`: Development only. `false`: base, QA, UAT, ProductionStaging, Production. |
| Result | `WalletUserProfile` row per user (`UserProfiles` container), plus Redis cache eviction | `RemittanceUserProfile` snapshot per user (ETag-guarded), plus Redis version/expire invalidation |

## 4.1 Wallet (`UserProfileChanged`)

1. **Validate.** The event is `InvalidEvent` if the profile is null, `eventId` or `occurredAtUtc` is missing, `occurredAtUtc` is more than `WalletUserProfileSync:MaximumFutureClockSkew` (5 min) ahead of Wallet's clock, `changedFields` is null, `schemaVersion` is outside 1..1, or `profileId` is not a non-empty GUID.

2. **Order.** The same `eventId` as the stored `LastEventId` is an idempotent replay (only sync metadata is refreshed). An `occurredAtUtc` earlier than the stored `LastSyncedAtUtc` is stale and dropped. The stored watermark is `min(occurredAtUtc, receive time)`.
3. **Apply.** The snapshot is authoritative: the whole projection is overlaid on the stored row, and `changedFields` is diagnostic only (names it does not recognise are logged and counted). It maps `tenantId`, names, `email`, `phoneNumber`, `trn`, `birthdate`, `nationality`, `timezone`, `gender`, `kycStatus`, `kycTier`, `gkoNumber`, `externalWalletReferenceNo`, `customerNameVariations`, `address`, `identificationDocument` and `prepaidCardInfo`. It ignores `status`, `cmsStatus`, `kycApplicantId`, `entraExternalId`, `occupation`, `occupationalLevel` and `eligibility`. It does not check that a wallet exists first. The read, decision and write run in one optimistic-concurrency loop, then the profile cache is evicted (skipped for stale events).
4. **Settle.** Applied, no-op, stale and replay complete the message. `InvalidEvent` and an unparsable body are dead-lettered (reasons `InvalidEvent`, `ParseFailure`). Any other exception abandons the message, and the broker redelivers it and dead-letters at the delivery limit.
5. **Dead-letter queue.** A separate processor settles it. A broker-originated (`Transient`) dead letter flags the user's profile as refresh-required and completes. A message Wallet refused itself is written to a quarantine store first, then the profile is flagged. If quarantine fails the message is retried with backoff up to `MaxQuarantineAttempts`, then completed and the profile flagged.
6. **Recovery path.** `WalletUserProfileResolver` reads a flagged or missing profile from Profile IAM over HTTP (change source `IamFetch`), so a lost event is repaired on the next read. If Profile IAM is down it serves the stored row.

## 4.2 Remittances (`UserProfileChanged` and `UserProfileUpdated`)

1. **Select.** A keyed mapping strategy per subject. An unknown subject (`UnsupportedEventType`), an unmappable body or a missing event id (`ParseFailure`) is dead-lettered. Any other failure propagates and the message is retried.
2. **`UserProfileChanged` is a partial update.** It maps names, `email`, `phoneNumber`, `trn`, `birthdate`, `gender`, `nationality`, occupation and level, `externalWalletReferenceNo`, `timezone`, `address` with proof of address, `identificationDocument` (`MISSING_DOCUMENT` counts as none) and the eligibility flags. `kycStatus` is parsed as an enum, falling back to `kycTier`. Eligibility is only trusted if `profile.eligibility` was present in the payload. An empty profile (no tenant, email or names) is rejected. `ApplyPartialChange` overlays these on the stored row; GKO number, birth country, proof-of-address validity, employment detail and the other fields the event lacks keep their stored values. It does not read `changedFields` or `schemaVersion`. With no stored row it writes the row at schema version 0, which the resolver treats as a miss and replaces with a live fetch from the GK One UserProfile API.
3. **`UserProfileUpdated` is a complete replacement.** The full `GkOneUser` goes through `GKOneUserModelDecorator` and the same `RemittanceUserProfileMapper` the API fetch uses, and

replaces the stored row. It has no occurrence time, so the message's enqueued time is used for ordering.

4. **Order and write.** The same `eventId` as the stored one, or an occurrence time at or before the stored one, is ignored. The write is ETag-guarded with up to 3 attempts and jittered backoff, then throws so the message is abandoned. After a write it stamps the Redis version key, then expires the profile key. A failed invalidation is logged and left to the cache expiry.

On a KYC completion, Remittances receives both events. `UserProfileUpdated` is enqueued right after the write, while the outbox's `UserProfileChanged` carries the older `occurredAtUtc` (the document's `UpdatedAtUtc`). If the full event is applied first, the partial one is dropped as older, which is harmless. If the full event's publish failed (errors are swallowed), the partial one applies on its own. This is from reading the code, not from a test.

### 4.3 Observations

1. **Nothing consumes `UserProfileChanged` end to end in Production.** IAM dispatch is off, Wallet's flag is off, and Remittances' flag is off. In QA and UAT, IAM and Wallet are on but Remittances is off, so the Remittances subscription is not read there and its backlog grows until the subscription TTL.
2. **`UserProfileUpdated` cannot be retired yet.** Remittances (and Order Management) still depend on it. Wallet already ignores it, and its ops doc says IAM must keep publishing it for that reason.
3. **Schema versioning is handled differently.** Wallet rejects any `schemaVersion` other than 1 and dead-letters it. Remittances ignores `schemaVersion`. A bump to 2 would stop Wallet and be applied silently by Remittances.
4. **The Subject label is part of the contract.** Both consumers match on it. Renaming a message type, or changing `MessageFilters`, breaks both subscriptions without any error on the producer.
5. **Remittances receives the sensitive `UserProfileUpdated` fields** (section 3.2) even though it maps only a subset of them.

## 5. Findings

1. **Production publishes no `UserProfileChanged`** while `EnableProfileChangeOutboxDispatch` is `false` there (section 2.2). Pending changes keep accumulating on the documents until it is switched on.
2. **`UserProfileUpdated` exposes sensitive fields** (section 3.2). It should be reviewed for what consumers actually need, and for whether `password`, `hash`, tokens, full card number and CVV may leave the service.
3. **`UsersService.UpdatePhoneNumber` is unreachable** in production code. Its `RecordProfileChange` site never runs. Only `ApplyVerifiedPhoneChangeAsync` publishes phone changes.

4. **The archived sample is stale.** `workbench/archive/feature-35690-userprofile-eventual-consistency/sample-user-profile-changed-event.json` in the IAM repo shows `paymentReferences.creditCard` and a `traceparent`-style `correlationId`. The shipped contract uses `prepaidCardInfo` and a bare trace id, and it adds `customerNameVariations`. Use the files in this folder instead.
5. **Duplicate Ondato recording code.** `OnDatoWebhookHandlingService` and `OndatoWebhookHandlingContext` each carry an identical private `RecordProfileChange` helper.